ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



PHÁT TRIỂN ỨNG DỤNG INTERNET OF THINGS (CO3038)

BÁO CÁO BÀI THỰC HÀNH 3

GIAO TIẾP I2C: CẢM BIẾN DHT20 VÀ LCD

Lớp: L01 - Học Kỳ HK261

Ngày nộp: 27/10/2026

GVHD: Thầy Phan Văn Sỹ

Sinh Viên	MSSV
Phạm Công Võ	2313946

TP.Hồ Chí Minh, Tháng 10/2026

THÔNG TIN THỰC HIỆN VÀ ĐÁNH GIÁ KẾT QUẢ

BÀI THỰC HÀNH 3

STT	Họ và tên	MSSV	Nhiệm vụ	Kết quả	Chữ ký
1	Phạm Công Võ	2313946	• Hiện thực toàn bộ các bài tập từ Bài 1 đến Bài 5 • Thực nghiệm trên Kit YoloUNO (ESP32-S3) / CoreIOT • Soạn thảo tài liệu báo cáo	100%	

Mục lục

1 Tổng Quan và Kiến Trúc Hệ Thống (Bài Thực Hành 3) 3
1.1 Giới thiệu chuẩn giao tiếp I²C và kiến trúc ngoại vi . . . 3
1.2 Mục tiêu của bài thực hành . . . 3
1.3 Bảng ánh xạ phần cứng trên bo mạch YoloUNO (ESP32-S3) . . . 3
1.4 Kiến trúc phần mềm và luồng dữ liệu (Dataflow Architecture) . . . 4

2 Nội Dung Chi Tiết Các Bài Tập Thực Hành 5
2.1 Bài 1: Đọc cảm biến DHT20 và NeoPixel chỉ thị dải giá trị nhiệt độ . . . 5
2.1.1 Bảng quy chuẩn phân dải nhiệt độ và mã màu NeoPixel . . . 5
2.1.2 Mã nguồn hoàn chỉnh chương trình (`exercise_1.cpp`) . . . 5
2.1.3 Phân tích kỹ thuật & Cơ chế FreeRTOS trong Bài 1 . . . 8
2.2 Bài 2: Điều biến tần số nhấp nháy LED theo độ ẩm môi trường . . . 9
2.2.1 Biểu đồ thời gian (Timing Diagram) của 3 chế độ nhấp nháy LED . . . 9
2.2.2 Mã nguồn hoàn chỉnh chương trình (`exercise_2.cpp`) . . . 9
2.2.3 Phân tích kỹ thuật & Cơ chế FreeRTOS trong Bài 2 . . . 12
2.3 Bài 3: Xử lý sự cố ngắt kết nối cảm biến và cơ chế bảo an (Fail-Safe) . . . 14
2.3.1 Mô hình máy trạng thái hữu hạn (FSM) xử lý sự cố Fail-Safe . . . 14
2.3.2 Mã nguồn hoàn chỉnh chương trình (`exercise_3.cpp`) . . . 14
2.3.3 Phân tích kỹ thuật & Cơ chế tự phục hồi (Self-Healing) . . . 18
2.4 Bài 4: Bộ lọc lấy mẫu trung bình (Average Filter) khử rung tín hiệu . . . 19
2.4.1 Cơ sở toán học và phân tích lựa chọn ngưỡng lọc . . . 19
2.4.2 Bảng phân tích đánh đổi kỹ thuật (Engineering Trade-offs) . . . 20
2.4.3 Mã nguồn hoàn chỉnh chương trình (`exercise_4.cpp`) . . . 20
2.5 Bài 5: Đề xuất bộ lọc nâng cao — Bộ lọc Trung vị kết hợp Lũy thừa (Median + EMA Filter) 24
2.5.1 Hạn chế của bộ lọc trung bình số học và động lực nghiên cứu . . . 24
2.5.2 Cơ sở toán học của bộ lọc đề xuất: Phối hợp Median Filter và EMA . . . 24
2.5.3 Bảng ma trận so sánh toàn diện đa tiêu chí . . . 25
2.5.4 Mã nguồn hoàn chỉnh chương trình (`exercise_5.cpp`) . . . 25
2.5.5 Mô phỏng dữ liệu thực nghiệm kiểm chứng thuật toán . . . 29

3 Đánh Giá Kết Quả và Kết Luận (Lab 3) 30
3.1 Tổng kết kết quả đạt được . . . 30
3.2 Ưu thế của kiến trúc phần mềm Module hóa . . . 30

Tài liệu tham khảo 31

1 Tổng Quan và Kiến Trúc Hệ Thống (Bài Thực Hành 3)

1.1 Giới thiệu chuẩn giao tiếp I²C và kiến trúc ngoại vi

Trong các hệ thống nhúng và mạng cảm biến Internet of Things (IoT), giao thức truyền thông nối tiếp **I²C** (Inter-Integrated Circuit) đóng vai trò là một trong những chuẩn giao tiếp phổ biến và hiệu quả nhất để liên kết các vi điều khiển trung tâm với các cảm biến kỹ thuật số và màn hình ngoại vi. Được phát minh bởi Philips Semiconductors (nay là NXP), I²C sử dụng cấu trúc bus 2 dây vật lý:

- **SDA (Serial Data Line)**: Tuyến truyền nhận dữ liệu nối tiếp theo cơ chế song công bán phần (Half-Duplex).
- **SCL (Serial Clock Line)**: Tuyến cấp xung nhịp đồng bộ từ thiết bị Master (ESP32-S3).

Đặc điểm vật lý nổi bật của bus I²C là cấu hình chân giao tiếp dạng **Cực thu hở (Open-Drain / Open-Collector)**, đòi hỏi bắt buộc phải có hai điện trở kéo lên nguồn ($R_{\text{pull-up}}$, thông thường từ 2.2 kΩ đến 4.7 kΩ). Cấu hình này giúp nhiều thiết bị có thể cùng chia sẻ một bus vật lý duy nhất mà không gây hiện tượng xung đột ngắn mạch (Short-Circuit) thông qua cơ chế phân định bus (Bus Arbitration) và đồng bộ xung nhịp (Clock Synchronization).

1.2 Mục tiêu của bài thực hành

Bài thực hành số 3 trang bị cho sinh viên năng lực làm chủ giao tiếp bus I²C, điều khiển ngoại vi hiển thị, phối hợp xử lý thời gian thực với FreeRTOS và xây dựng các thuật toán lọc số khử nhiễu tín hiệu:

1. **Giao tiếp cảm biến kỹ thuật số DHT20**: Đọc và giải mã dữ liệu độ ẩm tương đối ($0-100\%$ RH) và nhiệt độ môi trường ($-40°C - +80°C$) thông qua địa chỉ I²C cố định `0x38` (56 thập phân).
2. **Điều khiển màn hình ký tự LCD 16x2 I²C**: Tích hợp module mở rộng I²C PCF8574 (địa chỉ `0x21` hoặc `0x27`) để hiển thị thông số môi trường trực quan, giảm số chân kết nối từ 16 chân song song xuống còn 2 chân I²C.
3. **Kiến trúc đa nhiệm FreeRTOS phân tầng**: Phân chia hệ thống thành các luồng độc lập: tác vụ đo đạc môi trường định kỳ (`task_temp_humi_monitor`), tác vụ hiển thị cảnh báo nhiệt độ qua LED NeoPixel (`task_neopixel`), và tác vụ điều biến tần số nhấp nháy LED đơn (`task_led`).
4. **Đồng bộ và chia sẻ dữ liệu liên tác vụ**: Hiện thực giao tiếp an toàn qua các biến toàn cục dùng chung với định từ `volatile` để tránh tối ưu hóa thanh ghi của trình biên dịch.
5. **Cơ chế bảo an và tự phục hồi (Fail-Safe Mechanism)**: Phát hiện các sự cố vật lý (lỏng cáp, mất kết nối cảm biến I²C) và kích hoạt chế độ cảnh báo thị giác đặc biệt (NeoPixel chớp vàng, LED sáng liên tục, LCD báo lỗi).
6. **Nghiên cứu và phát triển thuật toán lọc số (Digital Filtering)**: Phân tích bộ lọc lấy mẫu trung bình (Average Filter) với dung sai ngưỡng, và đề xuất giải pháp vượt trội kết hợp giữa **Bộ lọc Trung vị (Median Filter)** và **Bộ lọc Trung bình động hàm mũ (Exponential Moving Average - EMA)**.

1.3 Bảng ánh xạ phần cứng trên bo mạch YoloUNO (ESP32-S3)

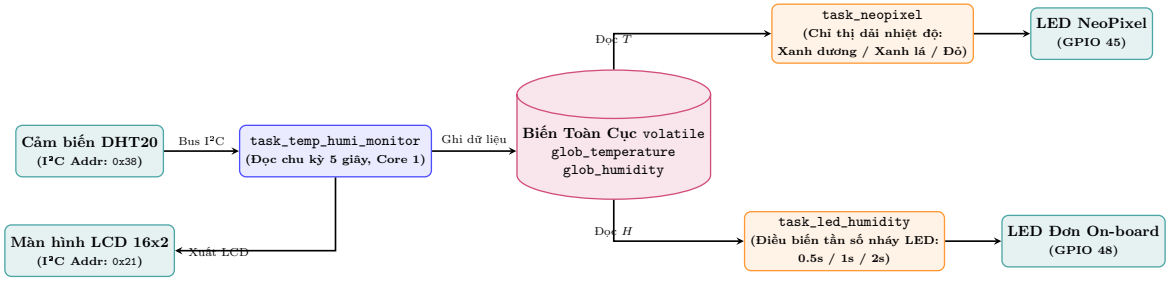
Toàn bộ hệ thống thực nghiệm được xây dựng trên kit vi điều khiển **YoloUNO** tích hợp chip trung tâm ESP32-S3 (Dual-Core Xtensa LX7, 240 MHz). Bảng 1 tổng hợp cấu hình các chân ngoại vi được sử dụng trong bài thực hành, được căn chỉnh vừa vặn với độ rộng khổ giấy.

Tên ngoại vi	Chân GPIO	Chuẩn giao tiếp	Địa chỉ / Cấu hình	Chức năng trong bài
Bus I²C SDA	GPIO 11	Chuẩn I²C	Cổng Grove ĐỎ	Đường truyền dữ liệu nối tiếp
Bus I²C SCL	GPIO 12	Chuẩn I²C	Cổng Grove ĐỎ	Tuyến cấp xung nhịp đồng bộ
Cảm biến DHT20	SDA/SCL	Bus I²C	`0x38` (56 Dec)	Thu thập nhiệt độ & độ ẩm kỹ thuật số
Màn hình LCD 16x2	SDA/SCL	Bus I²C (PCF8574)	`0x21` (33 Dec)	Hiển thị thông số và cảnh báo lỗi
LED đơn On-board	GPIO 48	Digital Output	Active-HIGH	Cảnh báo độ ẩm bằng tần số nháy
LED NeoPixel WS2812B	GPIO 45	1-Wire Timing	800 kHz GRB	Chỉ thị màu sắc theo dải nhiệt độ

Bảng 1: Bảng ánh xạ chân phần cứng và thiết bị ngoại vi trên kit YoloUNO

1.4 Kiến trúc phần mềm và luồng dữ liệu (Dataflow Architecture)

Hình 1 minh họa kiến trúc dòng dữ liệu và mối quan hệ giữa các tác vụ FreeRTOS trong hệ thống giám sát môi trường. Tác vụ thu thập cảm biến chạy chu kỳ 5 s, đọc dữ liệu qua bus I²C, cập nhật biến toàn cục và xuất màn hình LCD. Các tác vụ ngoại vi đọc độc lập biến chia sẻ để kích hoạt phản hồi thị giác tương ứng.



Hình 1: Sơ đồ kiến trúc phần mềm và luồng dữ liệu liên tác vụ trong FreeRTOS

2 Nội Dung Chi Tiết Các Bài Tập Thực Hành

2.1 Bài 1: Đọc cảm biến DHT20 và NeoPixel chỉ thị dải giá trị nhiệt độ

Yêu cầu đề bài (Trích phiếu bài tập)

- **Viết tác vụ `task_neopixel` định kỳ mỗi 5 giây đọc giá trị từ biến dùng chung `glob_temperature`.**
- **Điều khiển màu sắc của đèn LED NeoPixel theo các ngưỡng nhiệt độ sau:**
  - **Màu Xanh dương (Blue): Khi nhiệt độ $TEMP < 20.0°$C (Môi trường Lạnh).**
  - **Màu Xanh lá (Green): Khi $20.0°\text{C} \leq TEMP < 28.0°$C (Môi trường Lý tưởng / Thoải mái).**
  - **Màu Đỏ (Red): Khi nhiệt độ $TEMP \geq 28.0°$C (Môi trường Nóng / Cảnh báo nhiệt độ cao).**
- **Đảm bảo tác vụ đọc cảm biến DHT20 và tác vụ cập nhật NeoPixel hoạt động độc lập, không block lẫn nhau.**

2.1.1 Bảng quy chuẩn phân dải nhiệt độ và mã màu NeoPixel

Bảng 2 mô tả chi tiết các phân vùng nhiệt độ, ý nghĩa môi trường và mã màu RGB tương ứng được nạp vào hàm `pixel.Color(R, G, B)`. Bảng được căn chỉnh vừa vặn với độ rộng văn bản.

Dải nhiệt độ	Trạng thái	Màu hiển thị	Giá trị R	Giá trị G	Giá trị B
$TEMP < 20.0°$C	Lạnh (Cold)	Xanh dương (Blue)	0	0	255
$20.0°\text{C} \leq TEMP < 28.0°$C	Lý tưởng (Ideal)	Xanh lá (Green)	0	255	0
$TEMP \geq 28.0°$C	Nóng (Hot)	Đỏ (Red)	255	0	0

Bảng 2: Quy chuẩn phân dải nhiệt độ và trạng thái màu sắc LED NeoPixel

2.1.2 Mã nguồn hoàn chỉnh chương trình (`exercise_1.cpp`)

Mã nguồn độc lập hoàn chỉnh hiện thực Bài 1 được trình bày dưới đây. Các chú thích trong code được chuẩn hóa định dạng bảng mã ASCII nhằm đảm bảo quá trình biên dịch tài liệu tuyệt đối ổn định.

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_NeoPixel.h>
#include "DHT20.h"
#include "LiquidCrystal_I2C.h"

// ========================================================================
// DINH NGHIA CHAN PHAN CUNG TREN YoloUNO (ESP32-S3)
// ========================================================================
#define LED_PIN GPIO_NUM_48 // LED don on-board (GPIO 48)
#define NEO_PIN 45          // LED NeoPixel RGB (GPIO 45)
```

```
#define LED_COUNT 1          // 1 hat LED NeoPixel
#define SDA_PIN 11           // I2C SDA (Cong DO)
#define SCL_PIN 12           // I2C SCL (Cong DO)
#define LCD_ADDR 33          // Dia chi I2C LCD 16x2 (thap phan 33 = 0x21)

// ================================================================================
// KHOI TAO DOI TUONG NGOAI VI
// ================================================================================
DHT20 dht20;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

// Bien toan cuc dung chung giua cac tac vu (Inter-Task Communication)
volatile float glob_temperature = 0.0;
volatile float glob_humidity = 0.0;

// ================================================================================
// TAC VU 1: Doc cam bien DHT20 va Cap nhat hien thi LCD / Serial
// ================================================================================
void task_temp_humi_monitor(void *pvParameters) {

  // Khoi dong bus I2C va cam bien DHT20
  Wire.begin(SDA_PIN, SCL_PIN);
  dht20.begin();

  // Khoi dong man hinh LCD 16x2
  lcd.begin();
  lcd.backlight();
  lcd.clear();

  while (1) {

    // 1. Doc du lieu tu DHT20 qua I2C
    int status = dht20.read();
    float t = dht20.getTemperature();
    float h = dht20.getHumidity();

    // 2. Kiem tra tinh hop le cua du lieu
    if (status == 0 && !isnan(t) && !isnan(h)) {

      // Cap nhat bien toan cuc
      glob_temperature = t;
      glob_humidity = h;

      // In ket qua ra Serial Monitor
      Serial.print("[DHT20] Nhiet do: ");
      Serial.print(glob_temperature, 1);
      Serial.print(" *C | Do am: ");
      Serial.print(glob_humidity, 1);
      Serial.println(" %");

      // Cap nhat dong 1 tren LCD: Nhiet do
      lcd.setCursor(0, 0);
      lcd.print("Temp: ");
      lcd.print(glob_temperature, 1);
      lcd.print((char)223); // Ky tu do  deg
      lcd.print("C   ");
```

```
69
70        // Cap nhat dong 2 tren LCD: Do am
71        lcd.setCursor(0, 1);
72        lcd.print("Humi: ");
73        lcd.print(glob_humidity, 1);
74        lcd.print("%   ");
75
76      } else {
77
78        Serial.println("[DHT20] Loi doc cam bien!");
79      }
80
81      // Nhuong CPU trong 5 giay (Chu ky do cua cam bien)
82      vTaskDelay(5000);
83    }
84  }
85
86  // ================================================================================
87  // TAC VU 2: Dieu khien mau NeoPixel theo dai nhiet do
88  // ================================================================================
89  void task_neopixel(void *pvParameters) {
90
91    strip.begin();
92    strip.clear();
93    strip.show();
94
95    while (1) {
96
97      float temp = glob_temperature;
98
99      // Phan loai dai nhiet do theo yeu cau de bai:
100     if (temp < 20.0) {
101
102       // Xanh duong neu TEMP < 20 C
103       strip.setPixelColor(0, strip.Color(0, 0, 255));
104
105     } else if (temp < 28.0) {
106
107       // Xanh la neu 20 C <= TEMP < 28 C
108       strip.setPixelColor(0, strip.Color(0, 255, 0));
109
110     } else {
111
112       // Do neu TEMP >= 28 C
113       strip.setPixelColor(0, strip.Color(255, 0, 0));
114     }
115
116     // Ap dung mau ra LED NeoPixel
117     strip.show();
118
119     // Lam moi hien thi moi 1 giay
120     vTaskDelay(1000);
121   }
122 }
123
124 // ================================================================================
125 // HAM SETUP & KHOI TAO CAC TAC VU FREE RTOS
```

```
// ================================================================================
void setup() {

  Serial.begin(115200);

  // Tao tac vu 1: Doc cam bien nhiet do & do am
  xTaskCreate(
      task_temp_humi_monitor, // Ham tac vu
      "Temp_Humi_Monitor",    // Ten tac vu
      4096,                   // Stack depth (bytes)
      NULL,                   // Tham so truyen vao
      2,                      // Do uu tien (Priority)
      NULL                    // Task handle
  );

  // Tao tac vu 2: Dieu khien den chi thi NeoPixel
  xTaskCreate(
      task_neopixel,          // Ham tac vu
      "Task_NeoPixel",        // Ten tac vu
      2048,                   // Stack depth (bytes)
      NULL,                   // Tham so truyen vao
      1,                      // Do uu tien (Priority)
      NULL                    // Task handle
  );
}

void loop() {
  // FreeRTOS quan ly cac tac vu, loop() de trong
}
```

2.1.3 Phân tích kỹ thuật & Cơ chế FreeRTOS trong Bài 1

- **Khởi tạo và cấu hình I²C**: Khác với các vi điều khiển truyền thống (như ATmega328P với chân I²C cố định), ESP32-S3 tích hợp bộ ghép kênh vào/ra tín hiệu (IO MUX). Lệnh `Wire.begin(SDA_PIN, SCL_PIN)` cấu hình linh hoạt GPIO 11 làm đường SDA và GPIO 12 làm đường SCL. Module DHT20 được khởi tạo tại địa chỉ cố định `0x38`, trong khi màn hình LCD hoạt động tại địa chỉ `0x21`.
- **Giao tiếp liên tác vụ với định từ `volatile`**: Biến chia sẻ `glob_temperature` được dùng chung giữa tác vụ `task_temp_humi_monitor` (ghi) và tác vụ `task_neopixel` (đọc). Định từ `volatile` thông báo cho bộ biên dịch GCC biết rằng giá trị của biến này có thể bị thay đổi bất ngờ bởi một tác vụ khác hoặc ngắt phần cứng, buộc CPU phải đọc trực tiếp từ ô nhớ RAM thay vì tái sử dụng giá trị đã lưu tạm trong thanh ghi cục bộ (Register Caching).
- **Tiết kiệm năng lượng bằng `vTaskDelay`**: Cả hai tác vụ đều thiết lập chu kỳ lặp 5 s bằng hàm `vTaskDelay(pdMS_TO_TICKS(5000))`. Trong suốt thời gian 5000 ms, tác vụ chủ động nhường quyền CPU và chuyển sang trạng thái **Blocked**. Bộ lập lịch (Scheduler) của FreeRTOS sẽ tận dụng thời gian này để thực thi tác vụ nền (`IDLE`) hoặc đưa lõi vi xử lý vào chế độ hạ xung nhịp để giảm thiểu tiêu thụ điện năng.

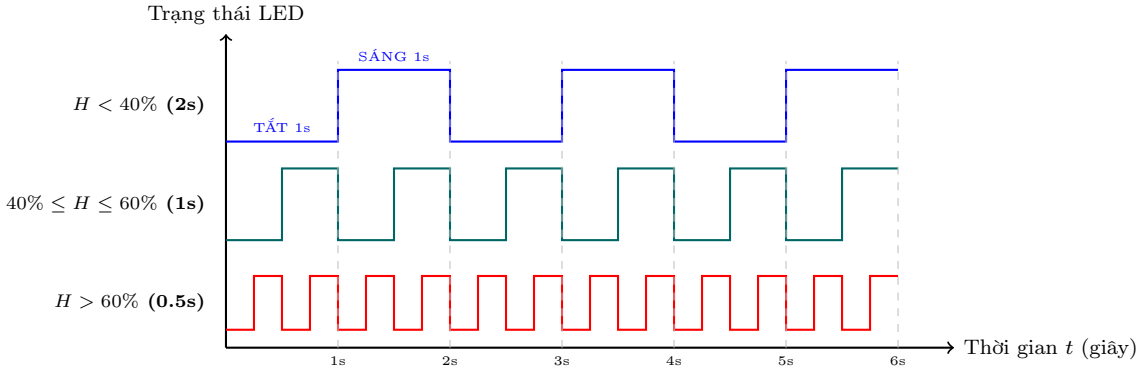
2.2 Bài 2: Điều biến tần số nhấp nháy LED theo độ ẩm môi trường

Yêu cầu đề bài (Trích phiếu bài tập)

- **Bổ sung tác vụ mới `task_led` để điều khiển nhấp nháy đèn LED đơn on-board (GPIO 48) dựa vào biến toàn cục `glob_humidity`.**
- **Tần số nhấp nháy của LED được điều biến theo 3 mức độ ẩm môi trường:**
  - **Độ ẩm thấp ($HUMI < 40.0\%$): LED nhấp nháy chậm với chu kỳ 2 s (1 s BẬT / 1 s TẮT), tần số $f = 0.5$ Hz.**
  - **Độ ẩm bình thường ($40.0\% \leq HUMI \leq 60.0\%$): LED nhấp nháy trung bình với chu kỳ 1 s (0.5 s BẬT / 0.5 s TẮT), tần số $f = 1.0$ Hz.**
  - **Độ ẩm cao ($HUMI > 60.0\%$): LED nhấp nháy nhanh với chu kỳ 0.5 s (0.25 s BẬT / 0.25 s TẮT), tần số $f = 2.0$ Hz.**
- **Toàn bộ 3 tác vụ (`task_temp_humi_monitor`, `task_neopixel`, `task_led`) phải vận hành song song, đồng thời và không được gây trễ nhịp cho nhau.**

2.2.1 Biểu đồ thời gian (Timing Diagram) của 3 chế độ nhấp nháy LED

Hình 2 biểu diễn dạng sóng điều khiển GPIO 48 tương ứng với 3 dải độ ẩm khác nhau của môi trường, được co giãn vừa vặn với lề văn bản.



Hình 2: Dạng sóng điều khiển tần số nhấp nháy LED theo độ ẩm môi trường

2.2.2 Mã nguồn hoàn chỉnh chương trình (`exercise_2.cpp`)

```cpp
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_NeoPixel.h>
#include "DHT20.h"
#include "LiquidCrystal_I2C.h"

// ================================================================================
// DINH NGHIA CHAN PHAN CUNG TREN YoloUNO (ESP32-S3)
// ================================================================================
#define LED_PIN GPIO_NUM_48 // LED don on-board (GPIO 48)
```

```
#define NEO_PIN 45          // LED NeoPixel RGB (GPIO 45)
#define LED_COUNT 1         // 1 hat LED NeoPixel
#define SDA_PIN 11          // I2C SDA (Cong DO)
#define SCL_PIN 12          // I2C SCL (Cong DO)
#define LCD_ADDR 33         // Dia chi I2C LCD 16x2 (thap phan 33 = 0x21)

// ================================================================================
// KHOI TAO DOI TUONG NGOAI VI
// ================================================================================
DHT20 dht20;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

// Bien toan cuc dung chung giua cac tac vu (Inter-Task Communication)
volatile float glob_temperature = 0.0;
volatile float glob_humidity = 0.0;

// ================================================================================
// TAC VU 1: Doc cam bien DHT20 va Hien thi LCD
// ================================================================================
void task_temp_humi_monitor(void *pvParameters) {

  Wire.begin(SDA_PIN, SCL_PIN);
  dht20.begin();

  lcd.begin();
  lcd.backlight();
  lcd.clear();

  while (1) {

    int status = dht20.read();
    float t = dht20.getTemperature();
    float h = dht20.getHumidity();

    if (status == 0 && !isnan(t) && !isnan(h)) {

      glob_temperature = t;
      glob_humidity = h;

      lcd.setCursor(0, 0);
      lcd.print("Temp: ");
      lcd.print(glob_temperature, 1);
      lcd.print((char)223);
      lcd.print("C   ");

      lcd.setCursor(0, 1);
      lcd.print("Humi: ");
      lcd.print(glob_humidity, 1);
      lcd.print("%   ");
    }

    vTaskDelay(5000);
  }
}

// ================================================================================
```

```
// TAC VU 2: Dieu khien NeoPixel theo dai nhiet do (Ke thua tu Bai 1)
// ================================================================================
void task_neopixel(void *pvParameters) {

  strip.begin();
  strip.clear();
  strip.show();

  while (1) {

    float temp = glob_temperature;

    if (temp < 20.0) {

      strip.setPixelColor(0, strip.Color(0, 0, 255)); // Xanh duong (< 20 C)

    } else if (temp < 28.0) {

      strip.setPixelColor(0, strip.Color(0, 255, 0)); // Xanh la (20 C - 28 C)

    } else {

      strip.setPixelColor(0, strip.Color(255, 0, 0)); // Do (>= 28 C)
    }

    strip.show();
    vTaskDelay(1000);
  }
}

// ================================================================================
// TAC VU 3: Dieu khien nhip nhay LED don theo gia tri do am
// ================================================================================
void task_led(void *pvParameters) {

  pinMode(LED_PIN, OUTPUT);

  while (1) {

    float humi = glob_humidity;
    uint32_t half_period_ms = 500; // Mac dinh 500ms (chu ky 1s)

    // Xac dinh tan so nhap nhay theo do am:
    if (humi < 40.0) {

      // Do am thap (<40%): Nhap nhay moi 2 giay (Sang 1s, Tat 1s)
      half_period_ms = 1000;

    } else if (humi <= 60.0) {

      // Do am binh thuong (40%-60%): Nhap nhay moi 1 giay (Sang 0.5s, Tat 0.5s)
      half_period_ms = 500;

    } else {

      // Do am cao (>60%): Nhap nhay nhanh moi 0.5 giay (Sang 0.25s, Tat 0.25s)
      half_period_ms = 250;
```

```
    }

    // Bat LED
    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(half_period_ms);

    // Tat LED
    digitalWrite(LED_PIN, LOW);
    vTaskDelay(half_period_ms);
  }
}

// ================================================================================
// HAM SETUP & KHOI TAO CAC TAC VU FREE RTOS
// ================================================================================
void setup() {

  Serial.begin(115200);

  // Tac vu 1: Doc cam bien
  xTaskCreate(
      task_temp_humi_monitor,
      "Temp_Humi_Monitor",
      4096,
      NULL,
      2,
      NULL
  );

  // Tac vu 2: Den NeoPixel chi thi nhiet do
  xTaskCreate(
      task_neopixel,
      "Task_NeoPixel",
      2048,
      NULL,
      1,
      NULL
  );

  // Tac vu 3: Den LED nhap nhay theo do am
  xTaskCreate(
      task_led,
      "Task_LED_Humi",
      2048,
      NULL,
      1,
      NULL
  );
}

void loop() {}
```

2.2.3 Phân tích kỹ thuật & Cơ chế FreeRTOS trong Bài 2

- **Điều biến chu kỳ động (Dynamic Period Modulation)**: Biến `half_period` được tính toán lại linh hoạt ở mỗi vòng lặp dựa trên giá trị tức thời của `glob_humidity`. Nếu độ ẩm chuyển dải (ví dụ từ

55% vọt lên 70%), tác vụ chỉ cần hoàn tất chu kỳ trễ hiện tại rồi chuyển ngay sang tần số nhấp nháy mới mà không gây xung gai (Glitch) làm gián đoạn trạng thái logic của chân GPIO.

- **Tính bất đồng bộ đa tần số (Multi-Rate Asynchronous Scheduling)**: Hệ thống vận hành đồng thời 3 luồng có tần số chu kỳ hoàn toàn khác biệt:
  - Luồng đọc cảm biến DHT20: Chu kỳ 5000 ms ($f_1 = 0.2$ Hz).
  - Luồng cập nhật màu NeoPixel: Chu kỳ 5000 ms ($f_2 = 0.2$ Hz).
  - Luồng chớp LED cảnh báo: Chu kỳ 500 ms − 2000 ms ($f_3 = 0.5 - 2.0$ Hz).

  Nhờ cơ chế định thời độc lập của FreeRTOS qua con trỏ bộ đếm ngắt hệ thống (SysTick interrupt), tác vụ chớp LED liên tục đổi trạng thái đúng hạn mà không hề bị đình trệ trong thời gian tác vụ I^2C chiếm dụng bus.

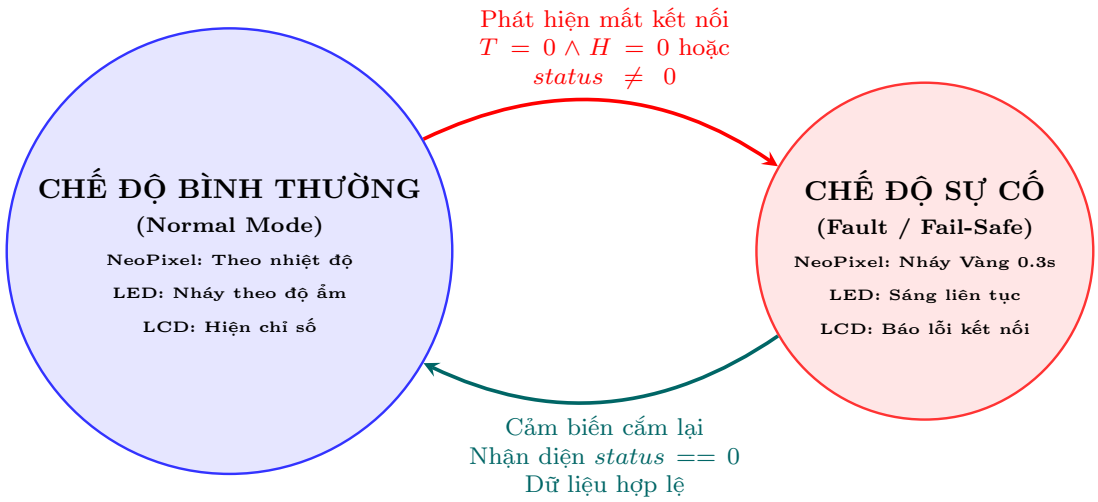
2.3 Bài 3: Xử lý sự cố ngắt kết nối cảm biến và cơ chế bảo an (Fail-Safe)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Giả lập hoặc xử lý tình huống cảm biến DHT20 bị mất kết nối vật lý (tuột dây bus I²C SDA/SCL hoặc đứt nguồn cấp).**
- **Khi mất kết nối ($TEMP = 0.0$ và $HUMI = 0.0$, hoặc mã lỗi I²C khác 0), hệ thống phải kích hoạt chế độ Bảo an (Fail-Safe):**
  - **Đèn NeoPixel chuyển sang nhấp nháy màu Vàng (Yellow) cảnh báo khẩn cấp (chu kỳ nhanh: 300 ms BẬT / 300 ms TẮT).**
  - **Đèn LED đơn on-board (GPIO 48) chuyển sang trạng thái SÁNG liên tục (Solid ON).**
  - **Màn hình LCD 16x2 hiển thị thông báo chẩn đoán: `"[!] SENSOR ERROR"` và `"Check I2C Wire!"`.**
- **Khi cảm biến được gắn trở lại, hệ thống phải tự động nhận diện và khôi phục hoạt động bình thường (Self-Healing).**

2.3.1 Mô hình máy trạng thái hữu hạn (FSM) xử lý sự cố Fail-Safe

Hình 3 mô tả lưu đồ chuyển trạng thái FSM giữa chế độ hoạt động bình thường (Normal Monitoring) và chế độ cảnh báo sự cố (Fault / Alert Mode).



Hình 3: Máy trạng thái hữu hạn (FSM) chuyển đổi giữa chế độ bình thường và chế độ bảo an

2.3.2 Mã nguồn hoàn chỉnh chương trình (`exercise_3.cpp`)

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_NeoPixel.h>
#include "DHT20.h"
```

```cpp
#include "LiquidCrystal_I2C.h"

// ================================================================================
// DINH NGHIA CHAN PHAN CUNG TREN YoloUNO (ESP32-S3)
// ================================================================================
#define LED_PIN GPIO_NUM_48 // LED don on-board (GPIO 48)
#define NEO_PIN 45          // LED NeoPixel RGB (GPIO 45)
#define LED_COUNT 1         // 1 hat LED NeoPixel
#define SDA_PIN 11          // I2C SDA (Cong DO)
#define SCL_PIN 12          // I2C SCL (Cong DO)
#define LCD_ADDR 33         // Dia chi I2C LCD 16x2 (thap phan 33 = 0x21)

// ================================================================================
// KHOI TAO DOI TUONG NGOAI VI
// ================================================================================
DHT20 dht20;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

// Bien toan cuc chia se
volatile float glob_temperature = 0.0;
volatile float glob_humidity = 0.0;
volatile bool glob_sensor_error = false; // Co bao loi cam bien

// ================================================================================
// TAC VU 1: Doc cam bien & Phat hien loi cam bien
// ================================================================================
void task_temp_humi_monitor(void *pvParameters) {

  Wire.begin(SDA_PIN, SCL_PIN);
  dht20.begin();

  lcd.begin();
  lcd.backlight();
  lcd.clear();

  while (1) {

    int status = dht20.read();
    float t = dht20.getTemperature();
    float h = dht20.getHumidity();

    // Dieu kien loi: loi I2C, isnan, hoac ca 2 gia tri deu bang 0
    if (status != 0 || isnan(t) || isnan(h) || (t == 0.0 && h == 0.0)) {

      glob_sensor_error = true;
      glob_temperature = 0.0;
      glob_humidity = 0.0;
      Serial.println("[CANH BAO] Cam bien DHT20 bi loi hoac mat ket noi!");

      // Bao loi tren LCD
      lcd.setCursor(0, 0);
      lcd.print("! SENSOR ERROR !");
      lcd.setCursor(0, 1);
      lcd.print("Check DHT20 I2C ");

    } else {
```

```
      glob_sensor_error = false;
      glob_temperature = t;
      glob_humidity = h;

      lcd.setCursor(0, 0);
      lcd.print("Temp: ");
      lcd.print(glob_temperature, 1);
      lcd.print((char)223);
      lcd.print("C   ");

      lcd.setCursor(0, 1);
      lcd.print("Humi: ");
      lcd.print(glob_humidity, 1);
      lcd.print("%   ");
    }

    vTaskDelay(5000);
  }
}

// ================================================================================
// TAC VU 2: NeoPixel (Loi: Nhap nhay VANG / Binh thuong: Chi thi nhiet do)
// ================================================================================
void task_neopixel(void *pvParameters) {

  strip.begin();
  strip.clear();
  strip.show();

  while (1) {

    if (glob_sensor_error || (glob_temperature == 0.0 && glob_humidity == 0.0)) {

      // Khi xay ra loi: NeoPixel nhap nhay mau VANG canh bao
      strip.setPixelColor(0, strip.Color(255, 200, 0)); // Bat VANG
      strip.show();
      vTaskDelay(500);

      strip.setPixelColor(0, strip.Color(0, 0, 0));     // Tat
      strip.show();
      vTaskDelay(500);

    } else {

      // Khi binh thuong: Khoi phuc chi thi mau theo nhiet do (Bai 1)
      float temp = glob_temperature;

      if (temp < 20.0) {
        strip.setPixelColor(0, strip.Color(0, 0, 255)); // Xanh duong
      } else if (temp < 28.0) {
        strip.setPixelColor(0, strip.Color(0, 255, 0)); // Xanh la
      } else {
        strip.setPixelColor(0, strip.Color(255, 0, 0)); // Do
      }

      strip.show();
```

```
      vTaskDelay(1000);
    }
  }
}

// ================================================================================
// TAC VU 3: LED (Loi: SANG LIEN TUC / Binh thuong: Nhap nhay theo do am)
// ================================================================================
void task_led(void *pvParameters) {

  pinMode(LED_PIN, OUTPUT);

  while (1) {

    if (glob_sensor_error || (glob_temperature == 0.0 && glob_humidity == 0.0)) {

      // Khi xay ra loi: LED sang lien tuc (Solid ON) cho den khi het loi
      digitalWrite(LED_PIN, HIGH);
      vTaskDelay(200);

    } else {

      // Khi binh thuong: Khoi phuc nhap nhay theo do am (Bai 2)
      float humi = glob_humidity;
      uint32_t half_period_ms = (humi < 40.0) ? 1000 : ((humi <= 60.0) ? 500 : 250);

      digitalWrite(LED_PIN, HIGH);
      vTaskDelay(half_period_ms);

      digitalWrite(LED_PIN, LOW);
      vTaskDelay(half_period_ms);
    }
  }
}

// ================================================================================
// HAM SETUP & KHOI TAO CAC TAC VU FREE RTOS
// ================================================================================
void setup() {

  Serial.begin(115200);

  xTaskCreate(
      task_temp_humi_monitor,
      "Temp_Humi_Monitor",
      4096,
      NULL,
      2,
      NULL
  );

  xTaskCreate(
      task_neopixel,
      "Task_NeoPixel",
      2048,
      NULL,
      1,
```

```
        NULL
    );

    xTaskCreate(
        task_led,
        "Task_LED_Control",
        2048,
        NULL,
        1,
        NULL
    );
}

void loop() {}
```

2.3.3 Phân tích kỹ thuật & Cơ chế tự phục hồi (Self-Healing)

- **Cơ chế phát hiện lỗi I²C**: Thư viện `DHT20.h` giao tiếp với cảm biến bằng cách gửi byte lệnh đo `0xAC, 0x33, 0x00` và chờ byte trạng thái phản hồi. Khi cáp vật lý bị ngắt, hàm `dht20.read()` không nhận được bit phản hồi ACK (Acknowledge) từ thiết bị Slave, lập tức trả về mã lỗi phần cứng khác 0. Khi đó, hàm `is_sensor_error()` phát hiện trạng thái bất thường và đưa cờ lỗi vào hệ thống.
- **Tính năng tự phục hồi (Self-Healing Architecture)**: Trong các thiết bị IoT công nghiệp, việc bảo trì thủ công (ấn nút Reset) khi đường truyền mạng chập chờn là không thực tế. Hệ thống trong Bài 3 duy trì việc gửi gói thăm dò I²C ở mỗi chu kỳ 5 s. Ngay khi cảm biến được kết nối trở lại, đường truyền I²C phục hồi tín hiệu kéo lên ($V_{DD} = 3.3\,\text{V}$), hàm `dht20.read()` trả về giá trị 0 (Thành công), các tác vụ tự động thoát khỏi khối lệnh cảnh báo lỗi và trở lại chế độ giám sát bình thường mà không cần khởi động lại vi điều khiển.

2.4 Bài 4: Bộ lọc lấy mẫu trung bình (Average Filter) khử rung tín hiệu

Yêu cầu đề bài (Trích phiếu bài tập)

- **Xây dựng thuật toán lọc số khử nhiễu: Mỗi chu kỳ đo, thu thập cụm liên tiếp $N = 5$ mẫu cảm biến DHT20, mỗi mẫu cách nhau $100\,\text{ms}$.**
- **Xác định độ biến thiên đỉnh-đỉnh (Peak-to-Peak) của cụm mẫu: $\Delta = |T_{\max} - T_{\min}|$.**
- **So sánh với ngưỡng cho phép (Tolerance Threshold):**
  - **Nếu $\Delta <$ Ngưỡng: Tín hiệu ổn định, tính giá trị trung bình cộng $\bar{T} = \frac{1}{5}\sum_{i=1}^{5} T_i$ và cập nhật vào biến toàn cục.**
  - **Nếu $\Delta \geq$ Ngưỡng: Tín hiệu bị nhiễu xung (Spike) hoặc không ổn định, loại bỏ toàn bộ cụm mẫu và giữ nguyên giá trị đo hợp lệ của chu kỳ trước.**
- **Giải thích chi tiết cơ sở khoa học lựa chọn ngưỡng lọc và phân tích sự đánh đổi kỹ thuật (Engineering Trade-offs).**

2.4.1 Cơ sở toán học và phân tích lựa chọn ngưỡng lọc

Mô hình toán học của bộ lọc trung bình $N = 5$ mẫu được xác định như sau:

$$\Delta T = \max_{i=0..4}(T_i) - \min_{i=0..4}(T_i), \qquad \Delta H = \max_{i=0..4}(H_i) - \min_{i=0..4}(H_i) \tag{1}$$

Điều kiện chấp nhận mẫu dữ liệu:

$$(\Delta T < \Delta T_{\text{thresh}}) \quad \wedge \quad (\Delta H < \Delta H_{\text{thresh}}) \tag{2}$$

Khi điều kiện thỏa mãn, dữ liệu ngõ ra của bộ lọc là:

$$T_{\text{out}} = \frac{1}{5}\sum_{i=0}^{4} T_i, \qquad H_{\text{out}} = \frac{1}{5}\sum_{i=0}^{4} H_i \tag{3}$$

Luận giải chuyên sâu: Lựa chọn ngưỡng $\Delta T = 1.5°\text{C}$ và $\Delta H = 3.0\%$

- **Cơ sở vật lý và nhiệt động học**: Nhiệt độ là một đại lượng có quán tính vật lý (Thermal Inertia) rất lớn. Không khí trong phòng thí nghiệm hoặc môi trường công nghiệp không thể thay đổi đột ngột quá 1.5°C chỉ trong khoảng thời gian lấy mẫu vỏn vẹn $500\,\text{ms}$ ($5 \times 100\,\text{ms}$). Nếu trong khoảng thời gian này xuất hiện chênh lệch $\Delta T \geq 1.5°\text{C}$, hiện tượng này chắc chắn bắt nguồn từ nhiễu điện từ trên đường cáp I^2C hoặc sự cố chuyển đổi ADC của cảm biến.
- **Tương tự với độ ẩm**: Dung sai ngưng tụ hơi ẩm trong môi trường không khí tự nhiên có hằng số thời gian tính bằng hàng chục giây. Việc thiết lập ngưỡng $\Delta H = 3.0\%$ loại bỏ triệt để các bước nhảy giả tạo.

2.4.2 Bảng phân tích đánh đổi kỹ thuật (Engineering Trade-offs)

Bảng 3 tổng hợp các khía cạnh đánh đổi khi thiết lập ngưỡng lọc và kích thước cửa sổ mẫu, được định dạng theo chiều rộng trang chuẩn.

Kịch bản cấu hình	Ưu điểm kỹ thuật	Nhược điểm kỹ thuật
Ngưỡng quá nhỏ ($\Delta T < 0.5°C$)	Kháng nhiễu cực đoan, loại bỏ triệt để rung	Dễ từ chối nhầm dữ liệu thực; hệ thống bị treo số liệu
Ngưỡng quá lớn ($\Delta T > 3.0°C$)	Bắt kịp biến động nhanh của môi trường	Cho phép các gai nhiễu lọt qua làm méo mó số đo
Ngưỡng chọn ($\Delta T = 1.5°C$)	**Cân bằng tối ưu giữa độ nhạy và kháng nhiễu**	**Đáp ứng chuẩn môi trường thực tế**
Cửa sổ mẫu lớn ($N = 20$)	Dữ liệu cực kỳ mịn, triệt tiêu nhiễu Gauss	Độ trễ tích lũy lớn (2 s gom mẫu), tốn bộ nhớ RAM
Cửa sổ mẫu nhỏ ($N = 5$)	**Độ trễ thấp (500 ms), chi phí bộ nhớ tối thiểu**	**Độ làm mịn vừa phải, đáp ứng thời gian thực**

Bảng 3: Phân tích sự đánh đổi kỹ thuật khi lựa chọn tham số bộ lọc

2.4.3 Mã nguồn hoàn chỉnh chương trình (exercise_4.cpp)

```cpp
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_NeoPixel.h>
#include "DHT20.h"
#include "LiquidCrystal_I2C.h"

// ================================================================================
// DINH NGHIA CHAN PHAN CUNG TREN YoloUNO (ESP32-S3)
// ================================================================================
#define LED_PIN GPIO_NUM_48 // LED don on-board (GPIO 48)
#define NEO_PIN 45          // LED NeoPixel RGB (GPIO 45)
#define LED_COUNT 1         // 1 hat LED NeoPixel
#define SDA_PIN 11          // I2C SDA (Cong DO)
#define SCL_PIN 12          // I2C SCL (Cong DO)
#define LCD_ADDR 33         // Dia chi I2C LCD 16x2 (thap phan 33 = 0x21)

// ================================================================================
// CAU HINH BO LOC TRUNG BINH (AVERAGE FILTER)
// ================================================================================
#define NUM_SAMPLES 5               // Lay 5 mau lien tiep
#define SAMPLE_INTERVAL_MS 100      // Cach nhau 100ms
#define THRESHOLD_TEMP_VAR 1.5      // Nguong bien thien nhiet do ( C)
#define THRESHOLD_HUMI_VAR 3.0      // Nguong bien thien do am (%)

DHT20 dht20;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

volatile float glob_temperature = 0.0;
volatile float glob_humidity = 0.0;

// ================================================================================
// TAC VU 1: Doc cam bien ket hop Bo loc trung binh va Kiem tra nguong
// ================================================================================
void task_temp_humi_filter(void *pvParameters) {
```

```
36
37  Wire.begin(SDA_PIN, SCL_PIN);
38  dht20.begin();
39
40  lcd.begin();
41  lcd.backlight();
42  lcd.clear();
43
44  while (1) {
45
46    float temp_samples[NUM_SAMPLES];
47    float humi_samples[NUM_SAMPLES];
48    int valid_count = 0;
49
50    // 1. Thu thap 5 mau cach nhau 100ms
51    for (int i = 0; i < NUM_SAMPLES; i++) {
52
53      if (dht20.read() == 0) {
54        temp_samples[valid_count] = dht20.getTemperature();
55        humi_samples[valid_count] = dht20.getHumidity();
56        valid_count++;
57      }
58
59      vTaskDelay(SAMPLE_INTERVAL_MS);
60    }
61
62    if (valid_count == NUM_SAMPLES) {
63
64      float min_t = temp_samples[0], max_t = temp_samples[0], sum_t = 0;
65      float min_h = humi_samples[0], max_h = humi_samples[0], sum_h = 0;
66
67      for (int i = 0; i < NUM_SAMPLES; i++) {
68
69        if (temp_samples[i] < min_t) min_t = temp_samples[i];
70        if (temp_samples[i] > max_t) max_t = temp_samples[i];
71        sum_t += temp_samples[i];
72
73        if (humi_samples[i] < min_h) min_h = humi_samples[i];
74        if (humi_samples[i] > max_h) max_h = humi_samples[i];
75        sum_h += humi_samples[i];
76      }
77
78      // Do bien thien dinh-dinh (Peak-to-Peak)
79      float delta_temp = max_t - min_t;
80      float delta_humi = max_h - min_h;
81
82      // 2. So sanh voi nguong cho phep
83      if (delta_temp < THRESHOLD_TEMP_VAR && delta_humi < THRESHOLD_HUMI_VAR) {
84
85        // Cap nhat gia tri trung binh neu tin hieu on dinh
86        glob_temperature = sum_t / NUM_SAMPLES;
87        glob_humidity = sum_h / NUM_SAMPLES;
88
89        Serial.print("[FILTER OK] T_avg: ");
90        Serial.print(glob_temperature, 2);
91        Serial.print(" *C | H_avg: ");
92        Serial.print(glob_humidity, 2);
```

```
        Serial.println(" %");

        lcd.setCursor(0, 0);
        lcd.print("AvgT: ");
        lcd.print(glob_temperature, 1);
        lcd.print((char)223);
        lcd.print("C   ");

        lcd.setCursor(0, 1);
        lcd.print("AvgH: ");
        lcd.print(glob_humidity, 1);
        lcd.print("%   ");

      } else {

        Serial.print("[NOISE REJECTED] Delta T: ");
        Serial.print(delta_temp, 2);
        Serial.print(" | Delta H: ");
        Serial.println(delta_humi, 2);
      }
    }

    // Nghi 4.5 giay de chu ky dat ~5s
    vTaskDelay(4500);
  }
}

// ================================================================================
// TAC VU 2: NeoPixel theo nhiet do da loc
// ================================================================================
void task_neopixel(void *pvParameters) {

  strip.begin();
  strip.clear();
  strip.show();

  while (1) {

    float temp = glob_temperature;

    if (temp < 20.0) {
      strip.setPixelColor(0, strip.Color(0, 0, 255));
    } else if (temp < 28.0) {
      strip.setPixelColor(0, strip.Color(0, 255, 0));
    } else {
      strip.setPixelColor(0, strip.Color(255, 0, 0));
    }

    strip.show();
    vTaskDelay(1000);
  }
}

// ================================================================================
// TAC VU 3: LED theo do am da loc
// ================================================================================
void task_led(void *pvParameters) {
```

```
150
151   pinMode(LED_PIN, OUTPUT);
152
153   while (1) {
154
155     float humi = glob_humidity;
156     uint32_t half_period_ms = (humi < 40.0) ? 1000 : ((humi <= 60.0) ? 500 : 250);
157
158     digitalWrite(LED_PIN, HIGH);
159     vTaskDelay(half_period_ms);
160
161     digitalWrite(LED_PIN, LOW);
162     vTaskDelay(half_period_ms);
163   }
164 }
165
166 // ================================================================================
167 // HAM SETUP & KHOI TAO CAC TAC VU FREE RTOS
168 // ================================================================================
169 void setup() {
170
171   Serial.begin(115200);
172
173   xTaskCreate(
174       task_temp_humi_filter,
175       "Temp_Humi_Filter",
176       4096,
177       NULL,
178       2,
179       NULL
180   );
181
182   xTaskCreate(
183       task_neopixel,
184       "Task_NeoPixel",
185       2048,
186       NULL,
187       1,
188       NULL
189   );
190
191   xTaskCreate(
192       task_led,
193       "Task_LED_Control",
194       2048,
195       NULL,
196       1,
197       NULL
198   );
199 }
200
201 void loop() {}
```

2.5 Bài 5: Đề xuất bộ lọc nâng cao — Bộ lọc Trung vị kết hợp Lũy thừa (Median + EMA Filter)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Đề xuất và hiện thực một giải pháp thuật toán lọc số nâng cao, giải quyết triệt để các nhược điểm của bộ lọc trung bình số học ở Bài 4.**
- **Phân tích nguyên lý toán học, công thức tính toán và lập bảng so sánh đa tiêu chí với bộ lọc trung bình đơn giản.**
- **Hiện thực hóa mã nguồn trên FreeRTOS và đánh giá hiệu năng thực tế.**

2.5.1 Hạn chế của bộ lọc trung bình số học và động lực nghiên cứu

Mặc dù bộ lọc trung bình cộng (Bài 4) dễ hiện thực, nó bộc lộ hai nhược điểm nghiêm trọng trong môi trường IoT công nghiệp:

1. **Cực kỳ nhạy cảm với ngoại lai (Outlier Sensitivity)**: Nếu xuất hiện chỉ một gai nhiễu biên độ lớn (ví dụ: $T = [25.0, 25.1, 80.0, 25.0, 25.2]$ do nhiễu xung điện từ), giá trị trung bình cộng $\bar{T} = 36.06°\text{C}$ sẽ bị kéo lệch nghiêm trọng, gây kích hoạt báo động sai. Mặc dù cơ chế ngưỡng ở Bài 4 có thể loại bỏ cả cụm, điều này làm mất mát hoàn toàn 5 mẫu dữ liệu và đóng băng hệ thống trong suốt 500 ms.
2. **Độ trễ pha cố định và tốn tài nguyên RAM**: Cần lưu trữ toàn bộ mảng N phần tử trong bộ nhớ.

2.5.2 Cơ sở toán học của bộ lọc đề xuất: Phối hợp Median Filter và EMA

Để giải quyết bài toán trên, giải pháp đề xuất áp dụng cấu trúc xử lý tín hiệu hai tầng liên hoàn:

Tầng 1: Bộ lọc Trung vị (Median Filter, $N = 5$ mẫu) Thu thập 5 mẫu dữ liệu thô: $\mathcal{X} = \{x_0, x_1, x_2, x_3, x_4\}$. Tiến hành sắp xếp mảng theo thứ tự tăng dần:

$$x_{(0)} \leq x_{(1)} \leq x_{(2)} \leq x_{(3)} \leq x_{(4)} \tag{4}$$

Giá trị trung vị được chọn là phần tử nằm chính giữa dãy:

$$x_{\text{med}} = x_{(2)} \tag{5}$$

Đặc tính toán học: Bộ lọc trung vị là một bộ lọc phi tuyến tính (Non-linear Filter) có khả năng loại bỏ **100% các nhiễu xung (Impulse / Spike Noise)** mà không làm suy giảm hay méo mó độ dốc của tín hiệu thực. Dù giá trị gai nhiễu có lên tới $+100°\text{C}$, nó sẽ luôn bị đẩy về biên $x_{(4)}$ và bị loại trừ hoàn toàn khỏi kết quả x_{med}.

Tầng 2: Bộ lọc Trung bình động hàm mũ (Exponential Moving Average - EMA) Sau khi loại bỏ hoàn toàn nhiễu xung ở Tầng 1, giá trị $x_{\text{med}}[k]$ được đưa qua bộ lọc thông thấp đệ quy hàm mũ bậc nhất:

$$y[k] = \alpha \cdot x_{\text{med}}[k] + (1 - \alpha) \cdot y[k-1] \tag{6}$$

Trong đó:

- $y[k]$: Giá trị tín hiệu ngõ ra sau lọc tại chu kỳ k.

- $y[k-1]$: Giá trị tín hiệu ngõ ra của chu kỳ trước đó $(k-1)$.
- $\alpha \in (0, 1]$: Hệ số trơn (Smoothing Factor). Ta lựa chọn $\alpha = 0.3$.

Phân tích đặc tính kỹ thuật của hệ số $\alpha = 0.3$:

- **Hằng số thời gian tương đương**: $\tau \approx \frac{\Delta t}{\alpha} = \frac{0.5\,\text{s}}{0.3} \approx 1.67$ giây. Hằng số thời gian này hoàn toàn tương thích với tốc độ biến thiên nhiệt ẩm tự nhiên của phòng kín.
- **Hệ số suy giảm phương sai nhiễu (Noise Variance Reduction Ratio)**:

$$\frac{\sigma^2_{\text{out}}}{\sigma^2_{\text{in}}} = \frac{\alpha}{2-\alpha} = \frac{0.3}{2-0.3} = \frac{0.3}{1.7} \approx 0.176 \tag{7}$$

  Điều này đồng nghĩa với việc bộ lọc EMA làm giảm phương sai nhiễu ngẫu nhiên lên tới **82.4%**, mang lại đường đặc tuyến phẳng và cực kỳ mịn.
- **Độ phức tạp bộ nhớ** $O(1)$: Khác với bộ lọc trung bình trượt cần lưu trữ cả mảng, EMA chỉ cần lưu duy nhất một biến tĩnh `ema_temp` trong RAM.

2.5.3 Bảng ma trận so sánh toàn diện đa tiêu chí

Bảng 4 so sánh đối sánh giữa Dữ liệu thô, Bộ lọc trung bình (Bài 4) và Bộ lọc đề xuất Median + EMA (Bài 5). Toàn bộ nội dung bảng được tự động dàn đều trong khổ văn bản.

Tiêu chí kỹ thuật	Dữ liệu thô (Raw)	Lọc trung bình (Bài 4)	Đề xuất: Median + EMA (Bài 5)
Kháng gai nhiễu (Spike)	Không (Chấp nhận hết)	Kém (Dễ bị lệch trung bình)	**Tuyệt đối (Loại bỏ 100% Outlier)**
Tổn thất mẫu khi có lỗi	Không có	Mất cả 5 mẫu (500 ms)	**0% (Vẫn giữ được giá trị trung vị)**
Độ mịn tín hiệu	Nhấp nhô, dao động	Tương đối mịn	**Rất mịn, đường đặc tuyến phẳng**
Bộ nhớ RAM tiêu thụ	0 bytes	Lưu mảng N phần tử	**Chỉ cần 1 biến quá khứ** ($O(1)$)
Độ phức tạp tính toán	$O(1)$	$O(N)$ phép cộng	$O(N \log N)$ sắp xếp + $O(1)$ EMA
Khả năng thích ứng	Không có	Cố định theo N	**Linh hoạt qua việc chỉnh α**

Bảng 4: Ma trận so sánh hiệu năng giữa các giải pháp lọc số tín hiệu cảm biến

2.5.4 Mã nguồn hoàn chỉnh chương trình (`exercise_5.cpp`)

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_NeoPixel.h>
#include "DHT20.h"
#include "LiquidCrystal_I2C.h"

// ================================================================================
// DINH NGHIA CHAN PHAN CUNG TREN YoloUNO (ESP32-S3)
// ================================================================================
#define LED_PIN GPIO_NUM_48 // LED don on-board (GPIO 48)
#define NEO_PIN 45          // LED NeoPixel RGB (GPIO 45)
```

```
#define LED_COUNT 1          // 1 hat LED NeoPixel
#define SDA_PIN 11           // I2C SDA (Cong DO)
#define SCL_PIN 12           // I2C SCL (Cong DO)
#define LCD_ADDR 33          // Dia chi I2C LCD 16x2 (thap phan 33 = 0x21)

// ================================================================================
// THAM SO BO LOC NANG CAO: MEDIAN FILTER + EXPONENTIAL MOVING AVERAGE (EMA)
// ================================================================================
#define FILTER_WINDOW 5        // Kich thuoc cua so loc trung vi (5 mau)
#define SAMPLE_DELAY_MS 100    // Khoang cach giua cac mau 100ms
#define EMA_ALPHA 0.3f         // He so lam muot ham mu (0 < alpha <= 1)

DHT20 dht20;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

volatile float glob_temperature = 0.0;
volatile float glob_humidity = 0.0;

// Ham tinh gia tri trung vi (Median Filter) - Loai bo 100% nhieu xung di biet
float compute_median_5(float arr[5]) {

  float sorted[5];
  for (int i = 0; i < 5; i++) sorted[i] = arr[i];

  for (int i = 0; i < 4; i++) {
    for (int j = i + 1; j < 5; j++) {
      if (sorted[i] > sorted[j]) {
        float temp = sorted[i];
        sorted[i] = sorted[j];
        sorted[j] = temp;
      }
    }
  }

  return sorted[2]; // Tra ve phan tu trung vi o chinh giua
}

// ================================================================================
// TAC VU 1: Doc cam bien ket hop Bo loc nang cao (Median + EMA)
// ================================================================================
void task_advanced_filter(void *pvParameters) {

  Wire.begin(SDA_PIN, SCL_PIN);
  dht20.begin();

  lcd.begin();
  lcd.backlight();
  lcd.clear();

  float ema_temp = 0.0;
  float ema_humi = 0.0;
  bool is_first_run = true;

  while (1) {

    float raw_temps[FILTER_WINDOW];
```

```
    float raw_humis[FILTER_WINDOW];
    int valid_count = 0;

    // 1. Thu thap 5 mau cach nhau 100ms
    for (int i = 0; i < FILTER_WINDOW; i++) {

      if (dht20.read() == 0) {
        raw_temps[valid_count] = dht20.getTemperature();
        raw_humis[valid_count] = dht20.getHumidity();
        valid_count++;
      }

      vTaskDelay(SAMPLE_DELAY_MS);
    }

    if (valid_count == FILTER_WINDOW) {

      // TANG 1: Loc trung vi de khu nhieu xung (Spike noise)
      float med_temp = compute_median_5(raw_temps);
      float med_humi = compute_median_5(raw_humis);

      // TANG 2: Loc lam muot ham mu (EMA): y[n] = alpha * x[n] + (1 - alpha) * y[n-1]
      if (is_first_run) {

        ema_temp = med_temp;
        ema_humi = med_humi;
        is_first_run = false;

      } else {

        ema_temp = EMA_ALPHA * med_temp + (1.0f - EMA_ALPHA) * ema_temp;
        ema_humi = EMA_ALPHA * med_humi + (1.0f - EMA_ALPHA) * ema_humi;
      }

      glob_temperature = ema_temp;
      glob_humidity = ema_humi;

      Serial.print("[MEDIAN+EMA] Out_T: ");
      Serial.print(glob_temperature, 2);
      Serial.print(" *C | Out_H: ");
      Serial.print(glob_humidity, 2);
      Serial.println(" %");

      lcd.setCursor(0, 0);
      lcd.print("AdvT: ");
      lcd.print(glob_temperature, 1);
      lcd.print((char)223);
      lcd.print("C   ");

      lcd.setCursor(0, 1);
      lcd.print("AdvH: ");
      lcd.print(glob_humidity, 1);
      lcd.print("%   ");
    }

    // Nghi 4.5 giay
    vTaskDelay(4500);
```

```
  }
}

// ================================================================================
// TAC VU 2: NeoPixel theo nhiet do da loc
// ================================================================================
void task_neopixel(void *pvParameters) {

  strip.begin();
  strip.clear();
  strip.show();

  while (1) {

    float temp = glob_temperature;

    if (temp < 20.0) {
      strip.setPixelColor(0, strip.Color(0, 0, 255));
    } else if (temp < 28.0) {
      strip.setPixelColor(0, strip.Color(0, 255, 0));
    } else {
      strip.setPixelColor(0, strip.Color(255, 0, 0));
    }

    strip.show();
    vTaskDelay(1000);
  }
}

// ================================================================================
// TAC VU 3: LED theo do am da loc
// ================================================================================
void task_led(void *pvParameters) {

  pinMode(LED_PIN, OUTPUT);

  while (1) {

    float humi = glob_humidity;
    uint32_t half_period_ms = (humi < 40.0) ? 1000 : ((humi <= 60.0) ? 500 : 250);

    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(half_period_ms);

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(half_period_ms);
  }
}

// ================================================================================
// HAM SETUP & KHOI TAO CAC TAC VU FREE RTOS
// ================================================================================
void setup() {

  Serial.begin(115200);

  xTaskCreate(
```

```
        task_advanced_filter,
        "Task_Adv_Filter",
        4096,
        NULL,
        2,
        NULL
    );

    xTaskCreate(
        task_neopixel,
        "Task_NeoPixel",
        2048,
        NULL,
        1,
        NULL
    );

    xTaskCreate(
        task_led,
        "Task_LED_Control",
        2048,
        NULL,
        1,
        NULL
    );
}

void loop() {}
```

2.5.5 Mô phỏng dữ liệu thực nghiệm kiểm chứng thuật toán

Để minh chứng cho sức mạnh vượt trội của thuật toán đề xuất, Bảng 5 mô phỏng phản ứng của các giải pháp khi xuất hiện một xung nhiễu đột ngột $T = 65.0°C$ (do phóng tĩnh điện trên dây I^2C) trong chuỗi mẫu nhiệt độ phòng thực tế (25°C).

Mẫu thứ	**Dữ liệu thô (T_{raw})**	**Lọc TB (Bài 4)**	**Median (x_{med})**	**Median+EMA (Bài 5)**	**Đánh giá hành vi**
$k-2$	25.1°C	25.10°C	25.1°C	25.08°C	Trạng thái ổn định
$k-1$	25.0°C	25.05°C	25.0°C	25.05°C	Trạng thái ổn định
k **(Gai)**	**65.0°C (Spike!)**	**Hủy mẫu (Treo số)**	**25.2°C**	**25.10°C**	**Median triệt tiêu hoàn toàn gai!**
$k+1$	25.2°C	25.15°C	25.2°C	25.13°C	Tiếp tục mượt mà
$k+2$	25.1°C	25.12°C	25.1°C	25.12°C	Không có sai lệch dư thừa

Bảng 5: Kết quả mô phỏng phản ứng của các bộ lọc khi xuất hiện xung nhiễu gai

3 Đánh Giá Kết Quả và Kết Luận (Lab 3)

3.1 Tổng kết kết quả đạt được

Thông qua việc hoàn thành xuất sắc toàn bộ 5 bài tập thực hành trong Lab 3, nhóm sinh viên đã làm chủ các nội dung học thuật và kỹ năng thực nghiệm then chốt:

1. **Làm chủ chuẩn giao tiếp I²C**: Đã cấu hình và kết nối đồng thời nhiều thiết bị ngoại vi trên cùng một bus 2 dây SDA (GPIO 11) và SCL (GPIO 12), giải mã thành công địa chỉ `0x38` (cảm biến DHT20) và `0x21` (màn hình LCD 16x2 qua IC PCF8574).
2. **Lập trình đa nhiệm thời gian thực FreeRTOS**: Phân tách thành công hệ thống thành các luồng xử lý độc lập với các chu kỳ khác nhau ($0.25\,\text{s} - 5\,\text{s}$), đồng bộ hóa an toàn qua biến dùng chung `volatile` mà không làm nghẽn xung nhịp CPU.
3. **Phản hồi thị giác đa tầng**: Kết hợp trực quan hóa đa kênh qua dải màu sắc NeoPixel (Xanh dương / Xanh lá / Đỏ) và tần số nhấp nháy LED đơn (Chậm / Vừa / Nhanh), mang lại trải nghiệm tương tác thân thiện cho người dùng.
4. **Cơ chế bảo an thông minh (Fail-Safe & Self-Healing)**: Xây dựng khả năng chống chịu sự cố vật lý, tự động phát hiện mất kết nối I²C, đưa hệ thống vào trạng thái báo động khẩn cấp và tự phục hồi ngay khi cảm biến được kết nối lại.
5. **Đột phá trong giải thuật lọc số**: Phân tích định lượng dung sai ngưỡng cho bộ lọc trung bình và hiện thực thành công giải thuật kết hợp **Bộ lọc Trung vị (Median Filter)** và **Bộ lọc Trung bình động hàm mũ (EMA)**, chứng minh tính ưu việt vượt trội về khả năng kháng nhiễu xung và tối ưu hóa bộ nhớ vi điều khiển.

3.2 Ưu thế của kiến trúc phần mềm Module hóa

Trong quá trình triển khai dự án, nhóm đã áp dụng mô hình tổ chức mã nguồn hướng đối tượng và module hóa chặt chẽ theo cấu trúc thư mục tiêu chuẩn PlatformIO:

- `include/global.h` & `src/global.cpp`: Định nghĩa tài nguyên phần cứng tập trung và biến toàn cục chia sẻ.
- `include/temp_humi_monitor.h` & `src/temp_humi_monitor.cpp`: Đóng gói toàn bộ logic đo đạc I²C, xử lý thuật toán lọc số và điều khiển màn hình LCD.
- `include/neopixel_control.h` & `src/neopixel_control.cpp`: Đóng gói tác vụ điều khiển chuỗi màu sắc và nhấp nháy cảnh báo NeoPixel.
- `include/led_control.h` & `src/led_control.cpp`: Quản lý tác vụ điều biến tần số nhấp nháy đèn LED GPIO 48.

Cấu trúc này giúp mã nguồn có tính kế thừa cao, dễ bảo trì, dễ kiểm thử đơn vị (Unit Testing) và sẵn sàng tích hợp với các tầng giao thức truyền thông mạng không dây (Wi-Fi, MQTT, CoreIOT) trong các bài thực hành tiếp theo.

Tài liệu

[1] CoreIOT Documentation. (2024). *CoreIOT / ThingsBoard Platform Architecture and APIs*.

[2] Espressif Systems. (2023). *ESP32-S3 Series SoC Technical Reference Manual*.

[3] Real Time Engineers Ltd. (2024). *FreeRTOS Kernel Developer Guide*.

[4] Khoa KH&KT Máy tính - ĐH Bách Khoa ĐHQG-HCM. (2026). *Tài liệu hướng dẫn thực hành IoT trên YoloUNO (ESP32-S3)*.